CampusStay: beginner Windows guide from installation to video

This guide is for Option B: Python Inventory/Reservation, PHP Payment/Notification, four XAMPP databases and an optional PHP browser console. Follow the stages in order. It is an Assignment 1 project; Docker/Minikube deployment is Assignment 2.

Use the ZIP named **CampusStay_Option_B_Python_PHP_XAMPP.zip**. All examples assume the extracted folder is **C:\Projects\campusstay_option_b** and XAMPP is **C:\xampp**. Replace paths if yours differ. Commands are for **Windows PowerShell**, not the Python prompt. Run one command at a time and wait for the prompt to return. If a command fails, resolve that step before continuing.

1. Understand what you will submit

Your institution requires a faculty-accessible GitHub repository, a design report PDF of at most 15 main pages excluding appendices, an 8-10 minute live demonstration video, evidence folders, genuine contributions/reflections and an AI-use declaration. Team size is at most four. The chosen extra challenge is **3C, saga with compensation**; Challenges 1 and 2 are mandatory.

The browser console is a small convenience. The video must also show real terminal/API calls and challenge scripts. The full installation process is longer than the final video: record the working application and its engineering demonstrations, not the entire software installation.

The initial code/documentation are substantially AI-assisted. Review and understand them, make genuine contributions, declare the assistance and write your own reflection. A similarity score cannot be guaranteed. Do not fabricate commits, author names, test screenshots or personal contributions.

2. Learn the few terms used in the guide

Term	Meaning here
Terminal / PowerShell	A window where you enter commands
Project root	The folder containing README.md, services, scripts, ui and docs
Python / PHP	Languages used to implement the services
XAMPP	Local database/PHP/Apache tools; its MySQL button starts MariaDB
Service	An independent program with an API and its own database
API	An HTTP interface another program can call
Port	A local address number: 8101-8104 for APIs, 8180 for the UI
Git	Tracks changes to files on your computer
GitHub	Hosts the repository for your team and faculty
Commit / push / clone	Save a change / upload commits / obtain a clean repository copy
.venv / .env	Python package environment / local service settings

3. Prepare folders and software

1. Open File Explorer. Open Local Disk C:. Create a folder named **Projects** if it does not exist.
2. Keep downloads in Downloads until the installer/extraction step. An installer is different from a project ZIP.
3. You need Git for Windows, 64-bit Python 3.12 and XAMPP. A code editor such as VS Code or Notepad is useful. OBS Studio is used later for recording.
4. Existing applications can remain in their current folders. CampusStay has its own project directory and project-specific databases/users.

4. Install Git for Windows

1. Open https://git-scm.com/downloads/win and download the Windows 64-bit installer.
2. Open the installer. Keep the standard options. When asked how Git should be available, choose the option that includes use from the command line/third-party software.

3. Keep Git Credential Manager enabled if the installer offers it. Choose an editor you recognise. Complete installation.
4. Close old terminal windows. Press the Windows key, type **PowerShell**, and open Windows PowerShell.
5. Enter the command below. Expect output starting with `git version`. The exact version can differ.

```
git --version
```

If Git is not recognised, reopen PowerShell after installation. If necessary, rerun the installer and check the command-line availability option.

5. Install Python 3.12, 64-bit

1. Visit https://www.python.org/downloads/windows/ and install the official Python install manager.
2. Open a new PowerShell window. Use the manager to install the 3.12 runtime. If 64-bit Python 3.12 already exists, start with the checks instead.
3. Other installed Python versions may remain. These project commands select 3.12 explicitly because the supplied Windows binary wheels target it.

```
pymanager install 3.12
py -3.12 --version
py -3.12 -c "import struct; print(struct.calcsize('P') * 8)"
```

Expected checks: **Python 3.12.x** and **64**. If the install-manager command is unavailable, reopen PowerShell and check the manager installation. A traditional Python 3.12 installation with the launcher also supports the two `py -3.12` checks. If you see >>>, you opened the Python interpreter; type `exit()` and return to PowerShell.

6. Install or check XAMPP

1. Visit https://www.apachefriends.org/download.html. Choose XAMPP for Windows with PHP 8; a PHP 8.2 package is the recommended local target. The application source is compatible with PHP 8.0+.
2. If you already use XAMPP, first check its PHP version with the command below. Reusing a compatible installation avoids replacing your existing application folders.
3. For a new installation, choose the installer and install in **C:\xampp**. Keep Apache, MySQL, PHP and phpMyAdmin components. Complete any required Visual C++ runtime installation requested by the installer.
4. Open **XAMPP Control Panel** from the Start menu or C:\xampp\xampp-control.exe.
5. Click **Start** beside **MySQL**. It should turn green. Note the database port, normally **3306**.

6. For phpMyAdmin, also start the control panel's main Apache component and open http://localhost/phpmyadmin/. The application itself uses separate Apache instances on 8103, 8104 and 8180, so its UI does not depend on port 80.

```
& 'C:\xampp\php\php.exe' -v
& 'C:\xampp\php\php.exe' -m
```

Expected: PHP 8.0 or later. The module list must contain **PDO**, **pdo_mysql** and **curl**. If pdo_mysql/curl is missing, open C:\xampp\php\php.ini in an editor, find the corresponding `extension=pdo_mysql` and `extension=curl` lines, remove a leading semicolon if present, save and rerun the module check. Restart application launchers after changing php.ini.

XAMPP calls its database component MySQL, while its bundled engine is MariaDB. The project uses its compatible SQL features. A separate installation can use another XAMPP folder/port; pass these values at the configuration step instead of overwriting an existing setup.

7. Create your GitHub account

1. Open https://github.com/. Create an account if needed and complete email verification.
2. Note your GitHub username. Each team member should use their own account and their own Git identity.
3. Decide which member/account will own the team repository. Others can be added as collaborators.
4. Use the repository visibility required by your course. A public repository is viewable without an invitation. For a private repository, grant the required faculty/reviewer accounts access before submission.

8. Extract and check the project folder

1. Download CampusStay_Option_B_Python_PHP_XAMPP.zip.
2. In File Explorer, right-click it and choose **Extract All**.
3. Move the inner **campusstay_option_b** folder to **C:\Projects**. Avoid accidentally keeping an extra outer ZIP-named folder in your command path.
4. Open the folder and check that README.md, services, scripts, ui, docs and windows are present.
5. Open PowerShell and enter:

```
Set-Location 'C:\Projects\campusstay_option_b'
Get-Location
Test-Path '.\services\inventory\run.py'
Test-Path '.\services\payment\src\main.php'
```

Expected: Get-Location shows your project root; both Test-Path commands print **True**. All later commands assume this current folder. Open another PowerShell window whenever instructed, and enter the same Set-Location command in it.

9. Configure your real Git identity

Replace the two examples below with your actual name and an email linked to your GitHub account. A GitHub-provided noreply email is also an option. Do not enter a classmate's name.

```
git config --global user.name "Your Actual Name"
git config --global user.email "your-verified-email@example.com"
git config --global --get user.name
git config --global --get user.email
```

The final two commands should print your own values. Git identity records who made a commit; it does not sign you into GitHub.

10. Create the local repository and baseline commit

From the project root:

```
git init
git branch -M main
git status
git add .
git status
git commit -m "Import AI-assisted CampusStay baseline for team review"
git log --oneline -1
```

The .gitignore excludes generated .env files, .env.admin, .runtime, logs and the virtual environment. The baseline message honestly identifies the starting point; it is not evidence that you individually wrote all the starter code. Later commits should describe your team's genuine review, tests, fixes and documentation work.

If .env or .env.admin appears staged, stop and check that you are using the supplied .gitignore before committing. Do not use `git add -f` for configuration or passwords. Generated private phpMyAdmin SQL is under the ignored .runtime folder.

11. Create an empty GitHub repository and push

1. In GitHub, use the **+** menu and **New repository**.
2. Enter **campusstay-microservices** as an example repository name. Set the course-appropriate visibility.
3. Keep the new repository empty: the project already contains its README and .gitignore.
4. Click **Create repository**. Copy the HTTPS repository address shown by GitHub.
5. Replace YOUR_USERNAME below with the actual owner, or paste the exact copied address.

```
git remote add origin https://github.com/YOUR_USERNAME/campusstay-microservices.git
git remote -v
git push -u origin main
```

Complete the browser sign-in if Git Credential Manager opens it. Then refresh the GitHub page: the source folders should appear. An account password is not the same as a Git authentication token. Use the browser-based sign-in offered by your credential manager rather than putting credentials into a repository URL.

If origin already exists, inspect `git remote -v`. Change it only if it points to the wrong repository:

```
git remote set-url origin `
  https://github.com/YOUR_USERNAME/campusstay-microservices.git
```

12. Create the Python package environment

Keep the current folder at the project root. Enter:

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe --version
```

The first command creates a .venv folder. The second should report Python 3.12.x. All package and application commands below use that environment's executable directly.

13. Install the supplied dependencies

Copy the complete two-line pip command, including the backtick at the end of its first line. The backtick continues a command in PowerShell.

```
.\.venv\Scripts\python.exe -m pip install --no-index `
  --find-links windows\wheels -r requirements-windows.txt
.\.venv\Scripts\python.exe -m pip check
```

Expected: installation succeeds; pip check ends with **No broken requirements found**. The supplied wheels install application/testing packages offline. The Python/XAMPP installers themselves are separate downloads. PHP services use built-in extensions and declare them in composer.json; there are no external PHP Composer packages to fetch.

If installation reports no matching wheel, verify Python 3.12 and 64-bit, then check that windows\wheels was extracted completely. Do not switch this environment to Python 3.14/3.15 while using the supplied 3.12 wheels.

14. Generate local service configuration

Ensure XAMPP MySQL is green. Run:

```
.\.venv\Scripts\python.exe scripts\configure.py
```

At the password prompt, enter your local XAMPP database root password. Characters are hidden. If that local root account has a blank password, press Enter. The program creates .env.admin plus a separate .env in each service and one in ui. The four service passwords are different and automatically generated. The UI configuration contains API addresses

and no database credentials.

For a different XAMPP location or database port, use your real values on the initial configuration instead:

```
.\.venv\Scripts\python.exe scripts\configure.py `
  --xampp D:\xampp --db-port 3307
```

Existing files are retained on later runs. To deliberately replace them, add `--force`, then rerun database provisioning so server passwords match the new service files. Do this while application services are stopped.

15. Create databases, users and tables

Run:

```
.\.venv\Scripts\python.exe scripts\init_databases.py
```

Expected: the engine version and four **Ready** lines. The program creates the following project databases and restricted accounts, applies each schema and grants runtime CRUD access only to the matching database.

Service	Database	Application user	API port
Inventory	campusstay_inventory	cs_inventory	8101
Reservation	campusstay_reservation	cs_reservation	8102
Payment	campusstay_payment	cs_payment	8103
Notification	campusstay_notification	cs_notification	8104

The application roles are restricted to connections from 127.0.0.1. The scripts do not drop project databases/tables. They reject reuse of a project account that has privileges outside its own database. Existing data is preserved; demo scripts create new room types with fresh IDs.

16. Inspect SQL files or use phpMyAdmin import

Each service has a database folder with its schema.sql, database-creation SQL and a user-creation template. To view tables, open phpMyAdmin and select the corresponding database. For example, campusstay_reservation contains bookings, booking_steps and idempotency_records.

The automated setup in step 15 is the simplest route. For a file import using the same generated passwords:

```
.\.venv\Scripts\python.exe scripts\init_databases.py --export-sql
```

Open **.runtime\setup_databases.local.sql** locally and import it in phpMyAdmin as the database administrator. It includes all four schemas and their matching application users. This generated file contains application passwords and is ignored by Git; keep it private. The service .env files generated in step 14 must remain consistent with it.

For fully manual setup instead, create each database with its own 00_create_database.sql; edit and run 01_create_user.sql.example; select that database and import schema.sql as administrator; then copy .env.example to .env and set the matching password. The Python generator handles quoting automatically and is preferred when passwords contain spaces or symbols.

17. Check the environment before startup

Run:

```
.\.venv\Scripts\python.exe scripts\doctor.py `
  --require-stopped --apache-check
```

Expected: Python/PHP version information, four database OK lines, free ports, Apache configuration syntax checks and **ENVIRONMENT CHECK PASSED**. The Apache checks create private per-service configurations under .runtime; the launchers do not modify XAMPP's global Apache configuration.

If a port is occupied, stop an earlier CampusStay launcher first. An XAMPP installation in another folder needs the correct XAMPP_HOME value. If Apache reports a missing runtime DLL, complete the Visual C++ runtime installation required by your XAMPP package.

18. Start the application in Terminal A

Open a PowerShell window named **Terminal A** for the application, enter the project folder and run:

```
Set-Location 'C:\Projects\campusstay_option_b'
.\.venv\Scripts\python.exe scripts\run.py
```

Wait for four service addresses and the browser UI address. Keep Terminal A open. It owns two Python service processes with two workers each, two independent PHP/Apache services and the PHP UI server. On Windows each PHP service uses its own Apache configuration/port with 32 request threads. These are local teaching settings, not a production capacity claim.

Open **http://127.0.0.1:8180/** in your browser. The four service indicators should become healthy. The optional XAMPP main Apache on port 80 is separate and can be used for phpMyAdmin.

19. Understand independent startup

The combined launcher is convenient; each service is still independently runnable. For manual startup, stop Terminal A with Ctrl+C first. Then open five PowerShell windows, enter the project folder in each, and run one command per window:

```
.\.venv\Scripts\python.exe services\inventory\run.py

.\.venv\Scripts\python.exe services\payment\run.py

.\.venv\Scripts\python.exe services\notification\run.py

.\.venv\Scripts\python.exe services\reservation\run.py

.\.venv\Scripts\python.exe ui\run.py
```

Keep these windows open. Use either the combined launcher or this independent method at a time. PHP business logic is in PHP files; the run.py in each PHP folder is only a standard-library launcher for XAMPP Apache. For concurrent Windows PHP demonstrations use this launcher, not PHP's single-worker Windows development server.

20. Check each service in the browser

Open these addresses after startup. They return JSON, not separate booking websites:

Check	Address
Inventory health	http://127.0.0.1:8101/health
Reservation health	http://127.0.0.1:8102/health
Payment health	http://127.0.0.1:8103/health
Notification health	http://127.0.0.1:8104/health
API specification	Add /openapi.json to any service address
Combined browser console	http://127.0.0.1:8180/

Expected health response: status ok and the correct service name. A service can be reachable yet unhealthy if its database is unavailable; inspect that service's terminal/log in that case.

21. Make your first booking through the UI

1. Open the console at http://127.0.0.1:8180/.
2. In Prepare a stay, keep a future stay date. Set the room name to a campus-visit example, capacity to 5 and price to 2500 INR.
3. Click **Create demo inventory**. Expect 5 of 5 rooms available and the new room type selected.
4. Enter a synthetic guest reference and the supplied example.test recipient. Keep payment successful; select Fail once, then retry for notification if demonstrating recovery.
5. Click **Book room**. Expect CONFIRMED, a booking ID, an INR amount and notification SENT.

6. Availability becomes 4 of 5. The workflow steps appear below.
7. Open Inspect the last API response to see the HTTP status, JSON and correlation ID. The UI is an API client; it has no direct database access.

22. Demonstrate replay, conflict and failure in the UI

1. After a successful booking, click **Replay original request**. The same original booking response returns; availability does not decrement again.
2. Click **Test changed-payload conflict**. It reuses the key with a changed guest reference. Expect HTTP 409.
3. Select Declined payment and click Book room. This is a new booking/key. Expect CANCELLED; compensation restores the held room, so availability returns to its pre-attempt value.
4. Select Slow successful payment and create another booking. Expect a 202 acceptance and then a refreshed current status of CONFIRMED. The original POST response remains 202; a subsequent GET reports the current state.
5. Use Inspect payment/notification to view the corresponding mock API response. No real financial or email transaction happens.

23. Use Terminal B for the scripted main flow

Keep Terminal A running. Open a second PowerShell window, **Terminal B**, enter the project folder and run:

```
Set-Location 'C:\Projects\campusstay_option_b'
Get-Date -Format o
.\.venv\Scripts\python.exe scripts\e2e.py
```

Expected: PASS for reservation, mock payment, confirmation and notification retry. The script creates its own synthetic inventory. You do not need to enter the UI booking ID into it. The printed timestamp and raw evidence files help support the live demonstration.

24. Run Challenge 1: concurrency

In Terminal B while services remain running:

```
Get-Date -Format o
.\.venv\Scripts\python.exe scripts\challenge_concurrency.py `
  --requests 20 --capacity 5
```

Expected: **five HTTP 201 successes and fifteen HTTP 409 rejections**, followed by PASS. Open evidence\challenge-1-concurrency\results.json and database_state.json to inspect the actual output/state. The atomic conditional update in Inventory prevents a negative count; checking a number in application memory would not safely arbitrate simultaneous requests.

25. Run Challenge 2: idempotency

```
Get-Date -Format o
.\.venv\Scripts\python.exe scripts\challenge_idempotency.py
```

Expected: first request plus three replays return the same result; changed payload returns 409; 20 identical simultaneous first requests create one booking, hold, charge and notification. Inspect evidence\challenge-2-idempotency. The script's evidence links show which files prove each claim.

26. Run Challenge 3C: saga compensation

```
Get-Date -Format o
.\.venv\Scripts\python.exe scripts\challenge_saga.py
```

Expected: a payment decline takes PENDING to RESERVED to PAYMENT_PENDING to COMPENSATING to CANCELLED. Capacity returns to one. Repeated compensation does not release twice, and the room can subsequently be booked. Inspect evidence\challenge-3c-saga.

This is the selected extra challenge. The supplementary outage checks are useful additional evidence; they do not claim completion of all optional Challenges 3A-3D.

27. Run the full suite and save your Windows evidence

1. In Terminal A press **Ctrl+C** and wait for Services stopped. If using five independent terminals, press Ctrl+C in each instead.
2. Keep XAMPP MySQL running.
3. In Terminal B run:

```
.\.venv\Scripts\python.exe scripts\doctor.py --require-stopped
.\.venv\Scripts\python.exe scripts\run_all.py
```

The suite starts and stops its own service processes. Expected: 35 Python test cases, 10 native PHP Payment cases and four native PHP Notification cases; then API/challenge/isolation/recovery PASS results and ALL CHECKS PASSED. Of the 49 unit/configuration cases, 40 cover business rules and nine cover launcher/configuration behavior.

Open evidence\tests\run_summary.json. It records the actual operating system, engine versions, result and source hashes. Your own Windows execution should record native_windows_run true. The supplied baseline evidence was executed on Linux with real PHP/MariaDB; it is not a substitute for your team's Windows recording.

28. Inspect the four databases

In phpMyAdmin, select one database at a time. Open SQL and run a read-only query appropriate to that database:

```
SELECT room_type_id, stay_date, capacity, available FROM room_stock;

SELECT id, status, amount_minor, notification_status FROM bookings;

SELECT booking_id, amount_minor, status FROM payments;

SELECT booking_id, status, attempts FROM notifications;
```

The queries belong to Inventory, Reservation, Payment and Notification respectively. An administrator can inspect all databases for diagnosis. Application services use their own restricted logins; the isolation test verifies all 12 peer-database connection attempts are denied.

29. Make genuine team contributions using branches

The institution considers real commits, contribution entries and each member's explanation. Each member should review their assigned service, make real improvements or tests they understand, and document actual observations. Use docs\EXPLAIN_AND_PERSONALISE.md as a workbook.

For a member's genuine change, first save current work and inspect status, then create a branch with a meaningful name:

```
git status
git switch main
git pull --ff-only
git switch -c review/inventory-yourname
```

Make your actual change in an editor, run its relevant tests and inspect the difference:

```
git diff
git add services/inventory docs
git diff --staged
git commit -m "Describe the real inventory change and validation"
git push -u origin review/inventory-yourname
```

Select only files relevant to your work; change the paths/branch/message for the service you actually worked on. Open a pull request on GitHub, have another member review it, merge it, then update main locally. Do not create meaningless changes or invented authors solely to manufacture activity.

```
git switch main
git pull --ff-only
git log --oneline --all -10
```

30. Commit your real evidence and documentation

After a successful run, inspect the files and commit the outputs your team actually generated:

```
git status
```

```
git add evidence docs
git diff --staged --stat
git commit -m "Record Windows validation and reviewed assignment documentation"
git push
```

If Git says nothing to commit, there is no new change to save. Do not fabricate a commit. The report, contribution table and reflection must agree with the work and evidence in the repository.

31. Test a fresh clone before submission

1. Stop the current application launcher. Keep the database server running.
2. Ask another member to perform a fresh clone if possible. A separate local folder is also useful for checking missing files.
3. From C:\Projects, clone the repository into a new folder. Replace the HTTPS address with your real repository.

```
Set-Location 'C:\Projects'
git clone https://github.com/YOUR_USERNAME/campusstay-microservices.git `
  campusstay_fresh
Set-Location 'C:\Projects\campusstay_fresh'
```

4. Repeat environment creation, dependency installation, configure, init_databases, doctor and run_all using that folder's README. Local .env files are intentionally not in Git, so a fresh clone needs configuration.
5. The fresh clone uses the same project database names by default. With the previous app stopped, provisioning aligns its service passwords to the fresh configuration and preserves existing data. When returning to an older checkout afterward, reprovision from that checkout's retained .env before starting it.
6. Record the fresh-clone check honestly in the report. A missing file/import should be fixed and committed, then the check repeated.

32. Complete the report and personal materials

1. Read docs\INSTITUTION_CHECKLIST.md and docs\CampusStay_Design_Report.pdf.
2. Run the following helper to enter your real team names, student IDs, service responsibilities, contribution summaries, repository URL and video URL. Initially leave the video URL empty if recording is still pending; enter it after upload.

```
.\.venv\Scripts\python.exe scripts\fill_submission_details.py
```

3. Each member writes their own half-page reflection under docs\reflections, using the provided prompts. Describe work actually performed, the hardest problem you personally solved and what you would change.
4. Review docs\AI_USE_DECLARATION.md. Retain accurate disclosure of substantial initial AI assistance and add your team's actual use/verification details.

5. To regenerate PDFs with your entered details and current test results, install the optional documentation dependencies and build:

```
.\.venv\Scripts\python.exe -m pip install -r requirements-docs.txt
.\.venv\Scripts\python.exe tools\build_documents.py
```

These optional PDF-building packages use an internet download; application packages in step 13 are bundled. Inspect the newly generated PDFs before committing them. Main report pages must remain at most 15 excluding appendices. Extra beginner/video guides are separate documents.

33. Prepare the video demonstration

Use docs\DEMO_VIDEO_GUIDE.md and its PDF. Choose who explains each service. Every team member must speak. Practise the main flow, declined payment and all three challenge commands before recording. Prepare one application terminal, one command terminal, the browser console, relevant evidence files and diagrams.

Make PowerShell text large enough to read in a recording. Use browser zoom if needed. Keep the recording focused on application behavior, commands and reasoning. Explain why the mechanisms work; reading PASS alone is insufficient.

34. Install OBS and make a short recording test

1. Download OBS Studio from https://obsproject.com/download and install it.
2. Open OBS and choose recording in its Auto-Configuration Wizard. For a modest laptop, a readable 1280x720, 30 fps recording is a practical starting point.
3. Add Display Capture as a source so you can show the browser and terminal. Select the correct microphone and check that its audio meter moves when you speak.
4. Record a 20-second sample using Start Recording, then Stop Recording. Open the saved recording and verify readable text and audible speech before the final take.

Use OBS's own quick-start reference for controls: https://obsproject.com/kb/quick-start-guide . Close unrelated windows and keep private .env/administrator settings outside the recording.

35. Record the required 8-10 minute video

Aim for approximately 9 minutes 30 seconds. Start the application before recording. Show:

Time	What to show
0:00-0:50	Actual team members, IDs and responsibility split
0:50-2:20	Architecture, four databases, language boundaries and direct API communication

Time	What to show
2:20-4:20	A successful booking, its state and notification retry
4:20-5:20	A declined payment, cancelled booking and restored availability
5:20-6:25	Challenge 1: 20 requests, five successes, final stock
6:25-7:30	Challenge 2: replay, changed payload and simultaneous first requests
7:30-8:35	Challenge 3C: state transitions and safe repeated compensation
8:35-9:30	Unit/evidence results, genuine Git history, trade-offs and AI-use disclosure

Use actual terminal/API calls, not only slides. Include a timestamp in screenshots of terminal evidence. All members should explain their real contribution. If a demonstration fails, fix it and record a truthful successful run; do not replace outputs with edited PASS text.

36. Save, upload and check the video link

Stop recording and watch the entire file. Check duration, audio for each member, readable commands, failure handling and no exposed credentials. If OBS saved MKV, use its Remux Recordings feature to produce MP4 if required; review the resulting file.

Upload the final video and supporting documents to the course-required location, including Google Drive as specified by the brief. Set viewing access so faculty can open them. Test the shared link using an appropriate viewer account or signed-out browser if it is a public link. Keep that exact link in the report and submission details. GitHub source/evidence is separate from the large video file.

37. Finalise the repository and submit

1. Enter the final video link with fill_submission_details.py and rebuild the report if needed.
2. Verify actual names, IDs, contribution table, reflections and AI declaration. Remove pending markers only when information is genuinely complete.
3. Check the report's main-page limit and video duration. Confirm all members speak.
4. Commit the final documents and your actual validation evidence, then push.
5. Open GitHub in the browser and confirm the latest files are present. Test faculty access to the repository and video.

6. Submit the GitHub repository link, final design report PDF and video link through the institution's required portal. Retain a copy/confirmation of what you submitted.

```
git status
git add docs evidence
git commit -m "Finalise reviewed report and verified demo links"
git push
git log --oneline -5
```

38. Stop and start the application on another day

To stop, press Ctrl+C in the combined launcher's terminal and wait for shutdown. If starting services individually, stop each of those terminals. Use the application's terminals for its servers; the XAMPP control panel manages its own components.

On the next day: start XAMPP MySQL, open PowerShell in the project root and run scripts\run.py with the project .venv executable. Open the UI at 8180. Reinstalling packages or recreating databases is unnecessary unless you intentionally changed the environment/configuration.

39. Troubleshooting reference

Symptom	Check and fix
File/path not found	Run Get-Location and Test-Path from step 8; locate the inner project folder.
py/git not recognised	Reopen PowerShell after installation and verify the selected tool.
Python shows >>>	Type exit() and run project commands in PowerShell.
No matching wheel	Use 64-bit Python 3.12 and the complete wheels folder.
PHP module missing	Check php.exe -m and enable pdo_mysql/curl in the actual XAMPP php.ini.
MySQL connection refused	Ensure XAMPP MySQL is running; verify its port.
Access denied	Check the administrator password for setup, and matching service passwords for runtime. Reprovision after deliberate regeneration.
Apache syntax/DLL error	Check XAMPP_HOME and run the supplied --apache-check. Complete the runtime requirements of your XAMPP installer.
Port occupied	Stop the previous launcher; use doctor to identify the port. Use one startup method at a time.
A service exits	Inspect its terminal and logs. Python logs are in root logs; PHP structured logs are in that service's logs/events.jsonl; Apache errors are in its .runtime/apache/error.log.
UI shows an unavailable service	Open its /health URL, check its terminal and database. Refresh status after fixing it.
UI asks to refresh	Refresh the page to obtain a current console session/token.
Tests cannot start services	Stop the normal application launcher before run_all. Keep only the database server running.
Git rejected push	Inspect the message. Verify the remote/login and pull team changes on a clean branch; do not force-push to hide history.
Report still says pending	Complete real submission details, reflections and links, then rebuild and inspect it.

40. Files to open when you want to understand the code

Read each service README, then its API entry point, rules and repository. In Reservation read saga.py and follow one booking ID through booking_steps. Compare the challenge's response records with the same booking IDs in each service's own snapshot. Use EXPLAIN_AND_PERSONALISE.md to practise explaining the choices in your own words.

Official references checked for this guide: Python Windows documentation, Apache Friends XAMPP Windows FAQ, Git/GitHub documentation and OBS quick-start. Installation screens and exact version numbers can vary; application commands and file paths refer to the supplied edition.